Pipeline Simulator: Build the AI System

An interactive workshop application for demonstrating when to use Jev decision nodes, LLM generation nodes, deterministic tools, and human review in a production AI system.

1. Workshop concept

Participants work in 5–6 teams to design an AI customer-support pipeline on a shared visual canvas. Each team starts with the same intentionally imperfect workflow and must improve it against latency, cost, safety, and automation constraints.

Core distinction: Jev nodes make bounded, typed decisions—such as routing, scoring, safety classification, and escalation—while LLM nodes generate or transform language. Deterministic tools validate rules and retrieve data; human-review nodes handle uncertain or high-risk cases.

2. How participants use it

- Open the app on a shared screen; give each team a laptop or tablet with the same canvas.
- Start every team from the same default pipeline: customer message → Jev intent router → LLM answer generator → policy check → customer response.
- Present the mission: automate routine tickets, keep p95 latency under 800 ms, stay within a cost budget, and prevent unsafe or low-confidence responses from reaching customers.
- Participants drag nodes from a palette, configure a small number of parameters, connect typed ports, and press Run traffic.
- The simulator replays an identical, hidden set of customer messages through every team's graph.
- Each edge animates as requests pass through it; each node displays request count, latency, cost, decisions or generated tokens, errors, and escalations.
- After the timed run, show a leaderboard and ask each team to explain which decisions it delegated to Jev, the LLM, tools, or humans.

3. Node model

Nodes are pre-defined components. Participants do not write node logic; they configure parameters and connect typed inputs and outputs.

Every node has a type, title, category, typed input ports, typed output ports, parameters, and an execution function.

```
type NodeDefinition = {
type: string;
title: string;
category: "jev" | "llm" | "tool" | "io";
inputs: Port[];
outputs: Port[];
params: Param[];
run: (input, params, context) => Promise<NodeResult>;
};
```

Port data types include text, json, enum, score, boolean, probability, image, and any. The UI blocks incompatible connections, such as connecting a boolean output directly to a text input.

4. Node catalog

Category	Purpose	Examples	Typical output
Input / output	Start or end workflow	Customer message, API request, response, final response	Text, JSON, queue

Category	Purpose	Examples	Typical output
Jev decision	Fast, bounded classification or scoring	Intent router, safety check, urgency scorer, escalation gate	Enum, score, boolean, probabilities
LLM generation	Open-ended language work	Draft answer, summarize thread, propose code fix	Text or structured JSON
Deterministic tool	Reliable action or validation	Policy engine, knowledge lookup, database query, PII redactor	JSON, boolean, text
Human review	Handle uncertainty or risk	Approval queue, agent handoff	Human decision or edited response

5. Core nodes

Node	Inputs	Outputs	Configurable parameters	Demonstrates
Customer message	None	text	Traffic mix, language, attack rate	Workload source
Jev intent router	text	enum, probabilities, confidence	Allowed intents, threshold	Fast routing
Jev safety check	text	boolean, confidence	Policy categories, threshold	Guardrails
Jev urgency scorer	text	score 1–5, confidence	Rubric levels, threshold	Calibrated prioritization
Jev escalation gate	confidence, score, boolean	boolean, confidence	Thresholds, fallback route	Decision policy
LLM answer generator	text, optional JSON	text	Model, temperature, max tokens, prompt	Generation cost and latency
LLM summarizer	text	text	Model, max tokens	Compression
Knowledge lookup	text	JSON	Source, top-k	Grounding
Policy engine	text, JSON	boolean, JSON	Rules, blocked phrases, data classes	Deterministic control
PII redactor	text	text	Entity types	Pre-processing
Human review queue	text, JSON	text	Capacity, reviewer latency	Cost of uncertainty
Customer response	text	None	Channel, SLA	Final output

6. Example: Jev router

The router receives customer text and asks Jev typed questions. It returns a selected choice, probability distribution, and confidence; low-confidence traffic can be routed to an escalation branch.

```
{
"state": { "customer_message": "My payment failed twice..." },
"questions": [
{ "type": "choice", "name": "intent",
"options": ["billing","technical_support","bug_report","feature_request"] },
{ "type": "score", "name": "urgency", "levels": [1,2,3,4,5] },
{ "type": "boolean", "name": "needs_human_review" }
]
}
```

7. Canvas interaction

- **Palette:** draggable node cards show name, category color, ports, explanation, and estimated latency/cost.
- **Canvas:** participants drag nodes, position them, connect ports, and configure them.
- **Connections:** edges validate data type, required inputs, acyclicity, and fan-out limits.
- **Inspector:** clicking a node opens parameters, schema preview, test button, cost/latency assumptions, and warnings.
- **Run controls:** Run traffic, Stop, Reset, Prompt-injection wave, and Traffic spike.
- **Live telemetry:** node counters, edge animations, latency, cost, escalations, and policy violations.

8. Application architecture

Recommended stack: React + TypeScript + React Flow for the canvas; FastAPI + Python for the backend; WebSockets for live execution events; SQLite or PostgreSQL for persistence; simple team-code authentication.

Browser canvas → FastAPI backend → Jev / LLM providers and mock tools. The backend validates graphs, executes simulated traffic, aggregates metrics, and streams node events to every participant.

9. Execution engine

- **Validation:** check node types, required inputs, port compatibility, cycles, start/end nodes, and safety coverage.
- **Scenario pack:** use a fixed mix of routine, complex, ambiguous, policy-violating, and prompt-injection messages.
- **Per-request trace:** record every node execution, output, confidence, latency, cost, and error.
- **Events:** emit node_started, node_completed, edge_transferred, and node_failed over WebSocket.
- **Metrics:** automation rate, escalation rate, unsafe-release rate, p50/p95 latency, cost per 1,000 requests, and decision quality.

10. Build plan

Phase 1 — Define the workshop contract: choose customer support, prepare 30–50 fixed messages, define scoring, create an imperfect default pipeline, and lock graphs during runs.

Phase 2 — Build the static canvas: create the React app, add React Flow, build node palette and custom Jev/LLM/tool/IO nodes, implement typed handles and connection validation, and add the inspector.

Phase 3 — Implement execution: create a node registry, standard NodeRunner interface, standard NodeResult, and deterministic mocks for Jev, LLM, policy, and knowledge lookup.

Phase 4 — Add real models: add server-side provider adapters, environment-variable credentials, timeouts, retries, caching controls, rate limits, spend caps, and redacted logging.

Phase 5 — Add simulation and telemetry: load scenarios, execute concurrently, emit WebSocket events, calculate rolling metrics, persist results, and show the leaderboard.

Phase 6 — Dry run: test default, weak, and strong pipelines; verify reconnect behavior; ensure graph state survives refresh; rehearse the 30-minute facilitator script.

11. Minimal first version

- Six core node types: customer message, Jev router, Jev safety check, LLM generator, policy engine, customer response.
- One human-review node.
- A fixed library of 30 support tickets.

- Draggable canvas with typed connections.
- Node parameter panel.
- Simulated latency and cost; no external API calls required initially.
- One Run traffic button.
- Live node counters, edge animation, and four metrics: p95 latency, cost per 1,000, automation rate, unsafe-release rate.
- Team leaderboard.

Facilitator takeaway

Use Jev for high-volume, bounded decisions; use the LLM only where generation is genuinely required; use deterministic tools and human review to make the system safe, economical, and observable.